



University of Pisa

Department of Computer Science
Bachelor's Degree

Bachelor's thesis

****here title****

****here title****

Supervisors:

Prof. Anna Bernasconi
University of Pisa

Dott. Giorgio dell'Immagine
****non lo so****

Candidate:

Sofia Scalzo
University of Pisa

Academic year 2025/2026

Abstract

Contents

Abstract	2
1 Mathematical Background	4
1.1 Finite Fields	4
1.1.1 The BabyBear Field	4
1.1.2 Roots of Unity	4
1.1.3 Field Extensions	5
1.2 Polynomials	5
1.2.1 The Number Theoretic Transform	5
1.2.2 Low-Degree Extension	5
1.2.3 Vanishing Polynomials	6
1.3 Reed-Solomon Codes	6
1.3.1 Relative Hamming Distance	6
1.4 Cryptographic Hash Functions	7
1.5 Merkle Trees	7
1.6 Proof Systems	7
1.6.1 Interactive Proofs	7
1.6.2 Arguments	8
1.6.3 The Fiat-Shamir Transformation	8
1.7 Interactive Oracle Proofs	8
1.8 The BCS Transformation	8
1.9 Polynomial Commitment Schemes	9
Bibliography	10

Chapter 1

Mathematical Background

This chapter introduces the notation and mathematical foundations used throughout the thesis.

1.1 Finite Fields

Definition 1.1 (Prime Field). Let p be a prime number. The *prime field* $\mathbb{F}_p$ is the set $\{0, 1, \dots, p-1\}$ equipped with addition and multiplication modulo p .

The multiplicative group $\mathbb{F}_p^* = \mathbb{F}_p \setminus \{0\}$ is cyclic of order $p-1$. An element $g \in \mathbb{F}_p^*$ is called a *generator* if $\{g^0, g^1, \dots, g^{p-2}\} = \mathbb{F}_p^*$. For any $a \in \mathbb{F}_p^*$, the multiplicative inverse can be computed via Fermat's little theorem:

$$a^{-1} \equiv a^{p-2} \pmod{p}. \quad (1.1)$$

1.1.1 The BabyBear Field

The implementation described in this thesis operates over the BabyBear prime field, defined by

$$p = 2^{31} - 2^{27} + 1 = 2\,013\,265\,921. \quad (1.2)$$

Since $p < 2^{31}$, every field element fits in a 32-bit unsigned integer, and the product of two elements fits in a 64-bit integer.

The multiplicative group $\mathbb{F}_p^*$ has order $p-1 = 2^{27} \cdot 15$. The factor 2^{27} , the *two-adicity* of the field, means that $\mathbb{F}_p^*$ contains a cyclic subgroup of order 2^{27} , which is essential for the Number Theoretic Transform.

1.1.2 Roots of Unity

Definition 1.2 (Primitive Root of Unity). An element $\omega \in \mathbb{F}_p$ is a *primitive N -th root of unity* if $\omega^N = 1$ and $\omega^k \neq 1$ for all $0 < k < N$.

Given a primitive N -th root of unity ω , the set $H = \{1, \omega, \omega^2, \dots, \omega^{N-1}\}$ is a multiplicative subgroup of $\mathbb{F}_p^*$ of order N . Because BabyBear has two-adicity 27, subgroups of order 2^k for any $k \leq 27$ exist and are generated from a fixed multiplicative generator g by taking $g^{(p-1)/2^k}$.

These subgroups serve as evaluation domains for the polynomials in the STARK protocol: the execution trace is evaluated on a subgroup H of order 2^h , and FRI operates on a larger domain.

1.1.3 Field Extensions

Definition 1.3 (Field Extension). A *field extension* $\mathbb{F}_{p^k}$ of degree k over $\mathbb{F}_p$ is the quotient ring $\mathbb{F}_p[X]/(f(X))$, where $f(X) \in \mathbb{F}_p[X]$ is an irreducible polynomial of degree k . Elements are polynomials of degree less than k with coefficients in $\mathbb{F}_p$.

In this work we use the *quartic extension* $\mathbb{F}_{p^4}$, constructed via the irreducible polynomial $X^4 - 11$. The constant 11 is chosen because it is not a fourth power in $\mathbb{F}_p$, which guarantees that $X^4 - 11$ is irreducible. An element of $\mathbb{F}_{p^4}$ is a tuple $(c_0, c_1, c_2, c_3) \in \mathbb{F}_p^4$ representing $c_0 + c_1X + c_2X^2 + c_3X^3$, with the reduction rule $X^4 = 11$.

This extension provides a field of approximately 2^{124} elements. It is used in the FRI protocol to sample the folding challenges: a challenge drawn from the base field $\mathbb{F}_p$ would offer only about 31 bits of soundness per round, whereas a challenge from $\mathbb{F}_{p^4}$ offers about 124 bits. Inversion in $\mathbb{F}_{p^4}$ uses Fermat's little theorem in the extension field, $a^{-1} = a^{p^4-2}$.

1.2 Polynomials

A univariate polynomial over $\mathbb{F}_p$ of degree at most d is $f(X) = \sum_{i=0}^d c_i X^i$ with $c_i \in \mathbb{F}_p$. It can be represented by its *coefficient vector* $(c_0, \dots, c_d)$ or by its *evaluation vector* on a domain D . Converting between the two representations, and evaluating a polynomial at a point, are the fundamental operations of the prover.

1.2.1 The Number Theoretic Transform

The conversion between coefficient and evaluation representations is performed using the Number Theoretic Transform (NTT), the finite-field analogue of the Fast Fourier Transform. When the evaluation domain is a multiplicative subgroup $D = \{1, \omega, \dots, \omega^{N-1}\}$ with N a power of 2, the NTT computes all evaluations $f(\omega^i)$ in $O(N \log N)$ operations. The algorithm exploits the decomposition

$$f(X) = f_{\text{even}}(X^2) + X \cdot f_{\text{odd}}(X^2),$$

recursing on the even- and odd-indexed coefficients over the squared subgroup, which has half the size. The inverse transform runs the same routine with ω^{-1} and divides by N .

1.2.2 Low-Degree Extension

The *low-degree extension* (LDE) of a polynomial is its evaluation on a domain larger than its coefficient count. Given the n coefficients of f , the LDE with blowup factor b evaluates f on a coset of a subgroup of size $b \cdot n$. The resulting evaluation vector is the Reed-Solomon codeword that the prover commits to. In our implementation the

low-degree extension is computed by padding the coefficients with zeros, applying a coset shift, and running the NTT on the enlarged domain. $D \supseteq H$ [Sta21, Section 3.4]

1.2.3 Vanishing Polynomials

Definition 1.4 (Vanishing Polynomial [Sta21, Section 5.1]). Given a set $S \subseteq \mathbb{F}$, the *vanishing polynomial* of S is $Z_S(X) := \prod_{s \in S} (X - s)$. When S is a multiplicative subgroup $H = \{1, \omega, \dots, \omega^{N-1}\}$, this simplifies to $Z_H(X) = X^N - 1$.

Z_H vanishes on every element of H and nowhere else. If a constraint polynomial $c(X)$ must be zero on H , then $Z_H(X)$ divides $c(X)$, and the quotient $q(X) = c(X)/Z_H(X)$ is a polynomial. The verifier checks that q has low degree, which certifies that c vanishes on H [Sta21]. This mechanism is the basis of the STARK construction presented in Chapter ??.

1.3 Reed-Solomon Codes

Definition 1.5 (Reed-Solomon Code [BS+18, Section 2.1]). For a finite field $\mathbb{F}$, a subset $S \subseteq \mathbb{F}$, and a rate parameter $\rho \in (0, 1)$, the *Reed-Solomon code* $\text{RS}[\mathbb{F}, S, \rho]$ is the set of functions $f : S \rightarrow \mathbb{F}$ that are evaluations of polynomials of degree less than $\rho|S|$:

$$\text{RS}[\mathbb{F}, S, \rho] = \{(f(x))_{x \in S} : f \in \mathbb{F}[X], \deg(f) < \rho|S|\}.$$

Each codeword is the evaluation of a polynomial of degree less than $\rho|S|$ on all points of S . The code has minimum distance $|S| - \rho|S| + 1$, so two distinct codewords differ on a large fraction of positions.

The parameter ρ is the *rate*, and $1/\rho$ is the *blowup factor*. In our implementation the blowup factor is 2 ($\rho = 1/2$): the codeword produced by the low-degree extension is twice the length of the coefficient vector.

1.3.1 Relative Hamming Distance

Definition 1.6 (Relative Hamming Distance). The *relative Hamming distance* between two vectors $u, v \in \mathbb{F}^n$ is

$$\Delta(u, v) = \frac{|\{i : u_i \neq v_i\}|}{n}.$$

The *distance* of a vector u from a code C is $\Delta(u, C) = \min_{v \in C} \Delta(u, v)$. A vector u is δ -close to C if $\Delta(u, C) \leq \delta$, and δ -far if $\Delta(u, C) > \delta$.

The FRI protocol tests proximity to a Reed-Solomon codeword. As stated in [BS+18, Section 2.1], the verifier's task is to distinguish between the case that $f \in \text{RS}[\mathbb{F}, S, \rho]$ and the case that f is far from $\text{RS}[\mathbb{F}, S, \rho]$ in relative Hamming distance.

1.4 Cryptographic Hash Functions

Definition 1.7 (Collision-Resistant Hash Function). A *collision-resistant hash function* is a function $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ such that no efficient algorithm can find distinct inputs $x \neq x'$ with $H(x) = H(x')$, except with negligible probability.

Hash functions serve two roles in STARK systems: constructing Merkle trees for committing to polynomial evaluations, and deriving the verifier’s challenges via the Fiat-Shamir heuristic. Our implementation uses two different hash functions for these two roles: SHA-256 for the Merkle commitments and Keccak- $f[1600]$ for the challenger.

The distinction matters for the Fiat-Shamir transformation. Chiesa and Orrù [CO25] prove that a *duplex sponge* construction (such as Keccak/SHA-3) is suitable for Fiat-Shamir, while Merkle-Damgård constructions (such as SHA-256) lack a comparable security proof in this setting. For this reason we use a Keccak-based sponge for the challenger, while retaining SHA-256 for the Merkle trees, where only collision resistance is required.

1.5 Merkle Trees

Definition 1.8 (Merkle Tree). A *Merkle tree* over a sequence $(v_0, v_1, \dots, v_{N-1})$ with $N = 2^k$ is a binary tree where each leaf stores the hash of one element, $\ell_i = H(v_i)$; each internal node stores the hash of the concatenation of its two children, $H(n_{\text{left}} \| n_{\text{right}})$; and the root r is a single digest that binds the entire sequence.

To prove that v_i is the value at position i , the prover provides an *authentication path*: the $\log_2(N)$ sibling hashes along the path from the leaf to the root. The verifier recomputes the root and accepts if it matches r . Verification requires $O(\log N)$ hash evaluations.

In the BCS transformation [BSCS16], Merkle trees commit to the oracle messages of the IOP: the prover evaluates a polynomial on the FRI domain, builds a Merkle tree over the evaluations, and sends the root to the verifier.

1.6 Proof Systems

We now establish the formal framework of proof systems, following Thaler [Tha22].

1.6.1 Interactive Proofs

Definition 1.9 (Interactive Proof [Tha22, Definition 3.1]). An *interactive proof system* for a function f is a pair $(\mathcal{P}, \mathcal{V})$ where $\mathcal{P}$ is a computationally unbounded prover and $\mathcal{V}$ is a probabilistic polynomial-time verifier. The protocol proceeds in rounds, and must satisfy:

- **Completeness.** For every input x , $\Pr[\mathcal{V} \text{ accepts}] \geq 1 - \delta_c$.
- **Soundness.** For every input x and every cheating prover $\mathcal{P}^*$ that sends a value $y \neq f(x)$, $\Pr[\mathcal{V} \text{ accepts}] \leq \delta_s$.

1.6.2 Arguments

Definition 1.10 (Argument System [Tha22, Definition 3.2]). An *argument system* for a function f is an interactive proof for f in which the soundness condition is only required to hold against prover strategies that run in polynomial time.

STARKs are *arguments*, not *proofs* in the formal sense: their soundness relies on the collision resistance of the hash function, which a computationally unbounded prover could break.

1.6.3 The Fiat-Shamir Transformation

A *non-interactive* proof consists of a single message from the prover to the verifier. The Fiat-Shamir transformation removes interaction from a public-coin protocol by replacing the verifier’s random challenges with the output of a hash function applied to the transcript so far. Under the random oracle model, this preserves soundness. Chiesa and Orrù [CO25] give a formal treatment using a duplex sponge, which resolves the ambiguity between writing to and reading from the transcript by using the sponge’s native *absorb* and *squeeze* operations. Our challenger implements this construction over Keccak- $f[1600]$.

1.7 Interactive Oracle Proofs

Definition 1.11 (Interactive Oracle Proof [BSCS16, Definition 2.2]). An *Interactive Oracle Proof* (IOP) for a language $\mathcal{L}$ is an interactive protocol between a prover $\mathcal{P}$ and a verifier $\mathcal{V}$ that proceeds in rounds. In each round:

1. the prover sends an *oracle message* π_i (a function that the verifier may query at chosen positions);
2. the verifier sends a random challenge r_i .

After k rounds, the verifier makes a bounded number of queries to $\pi_1, \dots, \pi_k$ and outputs *accept* or *reject*.

The key difference from a standard interactive proof is that the verifier has *oracle access* to the prover’s messages: it reads only a small number of positions rather than the full message. This is what enables succinct verification.

An IOP of *proximity* (IOPP) is an IOP in which the verifier tests whether the prover’s initial oracle is close to a codeword of a specified code, rather than testing membership in a language [Sta21].

1.8 The BCS Transformation

The BCS transformation [CO25] compiles a public-coin IOP into a non-interactive argument in the random oracle model:

1. **Commitment.** Each oracle message is committed using a Merkle tree; the prover sends the root.
2. **Challenge derivation.** Each challenge is derived from the transcript via Fiat-Shamir.
3. **Query answering.** The prover opens the requested positions together with their Merkle authentication paths.
4. **Verification.** The verifier checks the paths, recomputes the challenges, and runs the IOP decision procedure.

This is precisely the structure of our FRI commit phase: each folding layer is committed with a Merkle tree, its root is absorbed into the challenger, and the next folding challenge is squeezed from it. The query and verification steps are performed when the verifier opens the Merkle paths and checks folding consistency.

1.9 Polynomial Commitment Schemes

A *Polynomial Commitment Scheme* (PCS) consists of four algorithms:

- $\text{Setup}(\lambda, d) \rightarrow \text{pp}$: generates public parameters for polynomials of degree at most d .
- $\text{Commit}(\text{pp}, f) \rightarrow C$: produces a commitment C to the polynomial f .
- $\text{Open}(\text{pp}, f, z) \rightarrow (v, \pi)$: produces a proof π that $f(z) = v$.
- $\text{Verify}(\text{pp}, C, z, v, \pi) \rightarrow \{0, 1\}$: checks the evaluation proof.

The scheme must satisfy correctness and binding.

The STARK system implemented in this thesis uses a *FRI-based PCS* [BS+18]: a polynomial is committed via a Merkle tree over its low-degree extension, and evaluation proofs are constructed through the FRI low-degree testing protocol. This approach requires no trusted setup (transparent) and relies only on hash functions (post-quantum secure). The FRI protocol is described in detail in Chapter ??.

bibliography

- [BS+18] Eli Ben-Sasson et al. *Scalable, Transparent, and Post-Quantum Secure Computational Integrity*. Cryptology ePrint Archive, Report 2018/046. <https://eprint.iacr.org/2018/046>. 2018.
- [BSCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. “Interactive Oracle Proofs”. In: *Theory of Cryptography Conference (TCC)*. IACR ePrint 2016/116. <https://eprint.iacr.org/2016/116>. 2016.
- [CO25] Alessandro Chiesa and Eylon Orrù. *Fiat-Shamir via Sponges*. Cryptology ePrint Archive, Report 2025/536. <https://eprint.iacr.org/2025/536>. 2025.
- [Sta21] StarkWare. *ethSTARK Documentation*. Cryptology ePrint Archive, Report 2021/582. <https://eprint.iacr.org/2021/582>. 2021.
- [Tha22] Justin Thaler. *Proofs, Arguments, and Zero-Knowledge*. <https://people.cs.georgetown.edu/jthaler/ProofsArgsAndZK.html>. Foundations, Trends in Privacy, and Security, 2022.